

New

新版 C++ 特性

注意：考虑到算法竞赛的实际情况，本文将不会全面研究语法，只会讲述在算法竞赛中可能会应用到的部分。

本文语法参照 **C++11** 标准。语义不同的将以 **C++11** 作为标准，C++14、C++17 等语法视情况提及并会特别标注。

auto 类型说明符

`auto` 类型说明符用于自动推导变量等的类型。例如：

```
1 auto a = 1;           // a 是 int 类型
2 auto b = a + 0.1;     // b 是 double 类型
```

基于范围的 for 循环

下面是一种简单的基于范围的 `for` 循环的语法：

```
1 for (range_declaration : range_expression) loop_statement
```

上述语法产生的代码等价于下列代码（`__range`、`__begin` 和 `__end` 仅用于阐释）：

```
1 auto&& __range = range_expression;
2 for (auto __begin = begin_expr, __end = end_expr; __begin != __end; ++__begin) {
3     range_declaration = *__begin;
4     loop_statement
5 }
```

range_declaration 范围声明

范围声明是一个具名变量的声明，其类型是由范围表达式所表示的序列的元素的类型，或该类型的引用。通常用 `auto` 说明符进行自动类型推导。

range_expression 范围表达式

范围表达式是任何可以表示一个合适的序列（数组，或定义了 `begin` 和 `end` 成员函数或自由函数的对象）的表达式，或一个花括号初始化器列表。正因此，我们不应在循环体中修改范围表达式使其任何尚未被遍历到的「迭代器」（包括「尾后迭代器」）非法化。

这里有一个例子：

```
1 for (int i : {1, 1, 4, 5, 1, 4}) std::cout << i;
```

loop_statement 循环语句

循环语句可以是任何语句，常为一条复合语句，它是循环体。

这里有一个例子：

```

1 #include <LTiostream>
2
3 struct C {
4     int a, b, c, d;
5
6     C(int a = 0, int b = 0, int c = 0, int d = 0) : a(a), b(b), c(c), d(d) {}
7 };
8
9 int* begin(C& p) { return &p.a; }
10
11 int* end(C& p) { return &p.d + 1; }
12
13 int main() {
14     C n = C(1, 9, 2, 6);
15     for (auto i : n) std::cout << i << " ";
16     std::cout << std::endl;
17     // 下面的循环与上面的循环等价
18     auto&& __range = n;
19     for (auto __begin = begin(n), __end = end(n); __begin != __end; ++__begin) {
20         auto ind = *__begin;
21         std::cout << ind << " ";
22     }
23     std::cout << std::endl;
24     return 0;
25 }

```

初始化语句 (C++20)

在 C++20 中，范围循环中可以使用初始化语句：

```

1 #include <LTiostream>
2 #include <LTvector>
3
4 int main() {
5     std::vector<int> v = {0, 1, 2, 3, 4, 5};
6
7     for (auto n = v.size(); auto i : v) // the init-statement (C++20)
8         std::cout << --n + i << ' ';
9 }

```

函数对象

函数对象相较于函数指针，具有更高的灵活性，能够保存状态，也能够作为参数传递给其他函数。在函数中使用函数对象，仅需要将参数类型定义为模板参数，就能允许任意函数对象传入。

它不是一种语言特性，而是一种 [概念或者要求](#)，在标准库中广泛应用。

在通常的实现中，函数对象（FunctionObject）重载了 `operator()`，使得其实例能够像函数一样被调用，而 [lambda](#) 即作为一种典型的函数对象。

范围库 (C++20)

范围库是对迭代器和泛型算法库的一个扩展，使得迭代器和算法可以通过组合变得更强大，并且减少错误。

范围即可遍历的序列，包括数组、容器、视图等。

在需要对容器等范围进行复杂操作时，[范围库](#) 可以使得算法编写更加容易和清晰。

View 视图

视图是一种轻量对象，通过特定机制（如自定义迭代器）来实现一些算法，给范围提供了更多的遍历方式以满足需求。

范围库中已实现了一些常用的视图，大致分为两种：

1. **范围工厂**，用于构造一些特殊的范围工厂，使用这类工厂可以省去手动构造容器的步骤，降低开销，直接生成一个范围。
2. **范围适配器**，提供多种多样的遍历支持，既能像函数一样调用，也可以通过管道运算符 `|` 连接，实现链式调用。

范围适配器 作为 **范围适配器闭包对象**，也属于 **函数对象**，它们重载了 `operator|`，使得它们能够像管道一样拼装起来。

► 管道运算符

在复杂操作下，也能保持良好可读性，有以下特性：

若 A、B、C 为一些范围适配器闭包对象，R 为某个范围，其他字母为可能的有效参数，表达式

```
1 R | A(a) | B(b) | C(c, d)
```

等价于

```
1 C(B(A(R, a), b), c, d)
```

下面以 `ranges::take_view` 与 `ranges::iota_view` 为例：

```
1 #include <LTiostream>
2 #include <LTranges>
3
4 int main() {
5     const auto even = [](int i) { return 0 == i % 2; };
6
7     for (int i : std::views::iota(0, 6) | std::views::filter(even))
8         std::cout << i << ' ';
9 }
```

1. 范围工厂 `std::views::iota(0, 6)` 生成了从 0 到 5 的整数序列的范围
2. 范围适配器 `std::views::filter(even)` 过滤前一个范围，生成了一个只剩下偶数的范围
3. 两个操作使用管道运算符链接

上述代码不需要额外分配堆空间存储每步生成的范围，实际的生成和过滤运算发生在遍历操作中（更具体而言，内部的迭代器构造、自增和解引用），也就是零开销（Zero Overhead）。

同时，外部输入的范围生命周期，等同于 **范围适配器** 的内部元素的生命周期。如果外部范围（比如容器、范围工厂）已经销毁，那么再对这些的视图遍历，其效果与解引用悬垂指针一致，属于未定义行为。

为了避免上述情况，应该严格要求适配器的生命周期位于其使用的任何范围的生命周期内。

▼ 范围被销毁时，视图内元素均悬垂

```

1 #include <LTiostream>
2 #include <LTranges>
3 #include <LTvector>
4
5 using namespace std;
6
7 int main() {
8     auto view = [] {
9         vector<int> vec{1, 2, 3, 4, 5};
10        return vec | std::views::filter([](int i) { return 0 == i % 2; });
11    }();
12
13    for (int i : view) cout << i << ' '; // runtime undefined behavior
14
15    return 0;
16 }

```

Constrained Algorithm 受约束的算法

C++20 在命名空间 `std::ranges` 中提供大多数算法的受约束版本，可以用迭代器 - 哨位对或单个 `range` 作为实参来指定范围，并且支持投影和指向成员指针可调对象。另外还更改了大多数算法的返回类型，以返回算法执行过程中计算的所有潜在有用信息。

这些算法可以理解成旧标准库算法的改良版本，均为函数对象，提供更友好的重载和入参类型检查（基于 [concept](#)），让我们先以 `std::sort` 和 `ranges::sort` 的对比作为例子

```

1 #include <LTalgorithm>
2 #include <LTiostream>
3 #include <LTvector>
4
5 using namespace std;
6
7 int main() {
8     vector<int> vec{4, 2, 5, 3, 1};
9
10    sort(vec.begin(), vec.end()); // {1, 2, 3, 4, 5}
11
12    for (const int i : vec) cout << i << ", ";
13    cout << '\n';
14
15    ranges::sort(vec, ranges::greater{}); // {5, 4, 3, 2, 1}
16
17    for (const int i : vec) cout << i << ", ";
18
19    return 0;
20 }

```

`ranges::sort` 和 `sort` 的算法实现相同，但提供了基于范围的重载，使得传参更为简洁。其他的 `std` 命名空间下的算法，多数也有对应的范围重载版本位于 `ranges` 命名空间中。

使用这些范围入参，再结合使用上节视图，能允许我们在进行复杂操作的同时，保持代码可读性，让我们看一个例子：

```

1 #include <LAlgorithm>
2 #include <LTarray>
3 #include <LTiostream>
4 #include <LTranges>
5
6 using namespace std;
7
8 int main() {
9     const auto& inputs = views::iota(0u, 9u); // 生产 0 到 8 的整数序列
10    const auto& chunks = inputs | views::chunk(3); // 将序列分块, 每块 3 个元素
11    const auto& cartesian_product =
12        views::cartesian_product(chunks, chunks); // 计算对块自身进行笛卡尔积
13
14    for (const auto [l_chunk, r_chunk] : cartesian_product)
15        // 计算笛卡尔积下的两个块整数的和
16        cout << ranges::fold_left(l_chunk, 0u, plus{}) +
17            ranges::fold_left(r_chunk, 0u, plus{})
18            << ' ';
19 }

```

▼ 输出:

6 15 24 15 24 33 24 33 42

Lambda 表达式

请参考 [Lambda 表达式](#) 页面。

decltype 说明符

decltype 说明符可以推断表达式的类型。

```

1 #include <LTiostream>
2 #include <LTvector>
3
4 int main() {
5     int a = 1926;
6     decltype(a) b = a / 2 - 146; // b 是 int 类型
7     std::vector<decltype(b)> vec = {0}; // vec 是 std::vector <LTint> 类型
8     std::cout << a << vec[0] << b << std::endl;
9     return 0;
10 }

```

constexpr

另请参阅 [常量表达式 constexpr \(C++11\)](#)

std::function

▼ 请注意性能开销

std::function 会引入一定的性能开销, 经 [Benchmark](#) 测试, 通常会造成 2 到 3 倍以上的性能损失。

因为它使用了类型擦除的技术, 而这通常借由虚函数机制实现, 调用虚函数会引入额外的 [开销](#)。

请考虑使用 [Lambda 表达式](#) 或者 [函数对象](#) 代替。

std::function 是通用函数封装器, 定义于头文件 <LFunctional>。

`std::function` 的实例能存储、复制及调用任何 [可调用](#) 对象，这包括 [Lambda 表达式](#)、成员函数指针或其他 [函数对象](#)。

若 `std::function` 不含任何可调用对象（比如默认构造），调用时导致抛出 [std::bad_function_call](#) 异常。

```
1 #include <functional>
2 #include <iostream>
3
4 struct Foo {
5     Foo(int num) : num_(num) {}
6
7     void print_add(int i) const { std::cout << num_ + i << '\n'; }
8
9     int num_;
10 };
11
12 void print_num(int i) { std::cout << i << '\n'; }
13
14 struct PrintNum {
15     void operator()(int i) const { std::cout << i << '\n'; }
16 };
17
18 int main() {
19     // 存储自由函数
20     std::function<void(int)> f_display = print_num;
21     f_display(-9);
22
23     // 存储 Lambda
24     std::function<void()> f_display_42 = []() { print_num(42); };
25     f_display_42();
26
27     // 存储到成员函数的调用
28     std::function<void(const Foo&, int)> f_add_display = &Foo::print_add;
29     const Foo foo(314159);
30     f_add_display(foo, 1);
31     f_add_display(314159, 1);
32
33     // 存储到数据成员访问器的调用
34     std::function<int(Foo const&)> f_num = &Foo::num_;
35     std::cout << "num_: " << f_num(foo) << '\n';
36
37     // 存储到函数对象的调用
38     std::function<void(int)> f_display_obj = PrintNum();
39     f_display_obj(18);
40 }
```

std::tuple 元组

定义于头文件 `<tuple>`，即 [元组](#)，是 `std::pair` 的推广，下面来看一个例子：

```

1 #include <LTiostream>
2 #include <LTtuple>
3 #include <LTvector>
4
5 constexpr auto expr = 1 + 1 * 4 - 5 - 1 + 4;
6
7 int main() {
8     std::vector<int> vec = {1, 9, 2, 6, 0};
9     std::tuple<int, int, std::string, std::vector<int>> tup =
10         std::make_tuple(817, 114, "514", vec);
11     std::cout
12         << std::tuple_size_v<decltype(tup)> << std::endl; // 元组包含的类型数量
13
14     for (auto i : std::get<expr>(tup)) std::cout << i << " ";
15     // std::get<> 中尖括号里面的必须是整型常量表达式
16     // expr 常量的值是 3, 注意 std::tuple 的首元素编号为 0,
17     // 故我们 std::get 到了一个 std::vector<LTint>
18     return 0;
19 }

```

成员函数

函数	作用
<code>operator=</code>	赋值一个 tuple 的内容给另一个
<code>swap</code>	交换二个 tuple 的内容

例子

```

1 constexpr std::tuple<int, int> tup = {1, 2};
2 std::tuple<int, int> tupA = {2, 3}, tupB;
3 tupB = tup;
4 tupB.swap(tupA);

```

非成员函数

函数	作用
<code>make_tuple</code>	创建一个 tuple 对象, 其类型根据各实参类型定义
<code>std::get</code>	元组式访问指定的元素
<code>operator==</code>	等 按字典顺序比较 tuple 中的值
<code>std::swap</code>	特化的 std::swap 算法

例子

```

1 std::tuple<int, int> tupA = {2, 3}, tupB;
2 tupB = std::make_tuple(1, 2);
3 std::swap(tupA, tupB);
4 std::cout << std::get<1>(tupA) << std::endl;

```

可变参数模板

在 C++11 之前, 类模板和函数模板都只能接受固定数目的模板参数。C++11 允许任意个数、任意类型的模板参数。

可变参数函数模板

下列代码声明的函数模板 `fun` 可以接受任意个数、任意类型的模板参数作为它的模板形参。

```
1 template <typename... Values>
2 void fun(Values... values) {}
```

其中, `Values` 是一个模板参数包, `values` 是一个函数参数包, 表示 0 个或多个函数参数。函数模板只能含有一个模板参数包, 且模板参数包必须位于所有模板参数的最右侧。

所以, 可以这么调用 `fun` 函数:

```
1 fun();
2 fun(1);
3 fun(1, 2, 3);
4 fun(1, 0.0, "abc");
```

参数包展开

对于函数模板而言, 参数包展开的方式有以下几种:

1. 函数参数展开

```
1 f(args...);           // expands to f(E1, E2, E3)
2 f(&args...);          // expands to f(&E1, &E2, &E3)
3 f(n, ++args...);      // expands to f(n, ++E1, ++E2, ++E3);
4 f(++args..., n);      // expands to f(++E1, ++E2, ++E3, n);
5
6 template <typename... Ts>
7 void f(Ts...) {}
```

1. 初始化器展开

```
1 Class c1(&args...); // 调用 Class::Class(&E1, &E2, &E3)
```

1. 模板参数展开

```
1 template <class A, class B, class... C>
2 void func(A arg1, B arg2, C... arg3) {
3     tuple<A, B, C...>(); // 展开成 tuple<LTA, B, E1, E2, E3>()
4     tuple<C..., A, B>(); // 展开成 tuple<LTE1, E2, E3, A, B>()
5     tuple<A, C..., B>(); // 展开成 tuple<LTA, E1, E2, E3, B>()
6 }
```

递归展开

如果需要单独访问参数包中的每个参数, 则需要递归方式展开。

只需要提供展开参数包的递归函数, 并提供终止展开的函数重载。

举个例子, 下面这个代码段使用了递归函数方式展开参数包, 实现了可接受大于等于 1 个参数的取最大值函数。


```
1 // 递归终止函数
2 // C++20中, 使用 auto 也可以定义模板, 即「简写函数模板」
3 auto max(auto a) { return a; }
4
5 // 声明等价于
6 // template<typename T>
7 // auto max(T);
8
9 // 展开参数包的递归函数
10 auto max(auto first, auto... rest) {
11     const auto second = max(rest...);
12     return first > second ? first : second;
13 }
14
15 // 声明等价于
16 // template<typename First, typename... Rest>
17 // auto max(First, Rest...);
18
19 // int b = max(1, "abc");           // 编译不通过, 没有 > 操作符能接受 int 和
20 // const char* 类型
21 int c = max(1, 233);                // 233
22 int d = max(1, 233, 666, 10086);    // 10086
```

应用

在调试的时候有时会倾向于输出中间变量而不是 IDE 的调试功能。但输出的变量很多时, 就要写很多重复代码, 这时候就可以用上可变参数模板和可变参数宏。

```

1 // Author: Backlight, c0nstexpr(Coauthor)
2
3 #include <LTIostream>
4 #include <LTvector>
5
6 using namespace std;
7
8 template <typename T>
9 ostream& operator<<(ostream& os, const vector<T>& V) {
10     os << "[ ";
11     for (const auto& vv : V) os << vv << ", ";
12     os << "j";
13     return os;
14 }
15
16 namespace var_debug {
17 auto print(const char* fmt, const auto& t) {
18     for (; *fmt == ' '; ++fmt);
19     for (; *fmt != ',' && *fmt != '\\0'; ++fmt) cout << *fmt;
20     cout << '=' << t << *(fmt++) << '\\n';
21     return fmt;
22 }
23
24 void print(const char* fmt, const auto&... args) {
25     ((fmt = print(fmt, args)), ...); // C++17折叠表达式
26 }
27 // namespace var_debug
28
29 #define debug(...) var_debug::print(__VA_ARGS__, __VA_ARGS__)
30
31 int main() {
32     int a = 666;
33     vector<int> b({1, 2, 3});
34     string c = "hello world";
35
36     // before
37     cout << "manual cout print\\n"
38         << "a=" << a << ", b=" << b << ", c=" << c
39         << '\\n'; // a=666, b=[ 1, 2, 3, ], c=hello world
40     // 如果用printf的话, 在只有基本数据类型的时候是比较方便的, 然而是如果要输出vector等的内容的话, 就会比较麻烦
41
42     // after
43     cout << "vararg template print\\n";
44     debug(a, b, c); // a=666, b=[ 1, 2, 3, ], c=hello world
45
46     return 0;
47 }

```

这样一来, 如果事先在代码模板里写好 DEBUG 的相关代码, 后续输出中间变量的时候就会方便许多。

参考

1. [C++ 参考手册](#)

本页面最近更新: 2024/12/23 22:41:44, [更新历史](#)

发现错误? 想一起完善? [在 GitHub 上编辑此页!](#)

本页面贡献者: [alphagocc](#), [Backlight](#), [c0nstexpr](#), [CCXXXI](#), [ChenZ01](#), [cmpute](#), [CoderOJ](#), [Enter-tainer](#),

[FinParker](#), [gi-b716](#), [huanhuanonly](#), [lr1d](#), [ksyx](#), [StudyingFather](#), [Tiphereth-A](#), [Xeonacid](#), [zhib2000](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用